Introduction / Problem Statement

HyperLogLog is a solution to the count distinct (or cardinality estimation problem), where you want to find the number of distinct elements in a data stream that has distinct elements. The primary issue with this is that the naive solution would be to just iterate through the entire data stream (*O(n)* runtime), but requires using a set that could take *O(n)* space. This is impractical for large data sets. HyperLogLog is an algorithm to estimate the unique elements in a multi-set with *O(1)* time to add to the multiset and *O(log(log(n))* space.

Overall Intuitive Approach / Solution

The intuition behind HyperLogLog is that you can estimate the cardinality of the multi-set (with uniformly distributed random numbers) using the maximum number of leading zeroes of the binary representation of each number in the multi-set. For example, think of flipping a coin. If you got 3 heads, you would assume that you had to flip the coin around 8 times. So, applying that intuition to binary, if you had a uniformly distributed set (of fixed universe size) of binary numbers, imagine you have 0b0001xx…, representing a binary number with 3 leading 0s. At each digit from the left, you have a ½ chance of each digit being 0, so you can assume that since 0b0001xx… appeared, you must have observed at least 8 (2^3) elements. More generally, if the number with the maximum number of leading 0s has n leading 0s, then you would estimate there to be around 2^n distinct elements. To generalize HyperLogLog to more than just numbers, if you had a multi-set that wasn't just uniformly distributed random numbers, you would use a hash function to create a new multi-set of uniformly distributed random numbers. Using the estimation on the new multi-set should give you an estimate of the number of distinct elements in the original multi-set, as the new hashed multi-set should have the same number of distinct elements.

Implementation

The initial simple approach consists of just iterating through the uniformly distributed (and hashed) multi-set, keeping a variable to keep track of the element's hash that has the highest number of leading 0s. Then, once you finish iterating through all the numbers, you count the number of leading 0s (call it n), and return 2^n as the estimation of the distinct elements. However, this simple approach incurs a lot of variance, so we can further optimize it by splitting each set of the multi-set into subsets and having several counters for them. In particular, for some specified precision constant b, we create $m=2^b$ registers and each register will keep track of the longest prefix of leading 0s for the elements assigned to it. Each element is assigned to a register based on the first b bits, then the rest of the bits are used to count leading 0s. We take the harmonic mean of the estimates each register produces to get an overall estimate and also multiply by a correction constant α_m that accounts for hash collisions. α_m is around 0.7 and depends on m.

Below is the pseudocode for HyperLogLog from the original Google research paper; in phase 2, there is some extra code to correct for particularly small or large cases, but in most cases, the formula on line 10 is used for estimation.

Require: Let $h : \mathcal{D} \to \{0,1\}^{32}$ hash data from domain $\mathcal{D}$.
Let $m = 2^p$ with $p \in [4..16]$.

Phase 0: Initialization.

Define $\alpha_{16} = 0.673$, $\alpha_{32} = 0.697$, $\alpha_{64} = 0.709$,
$\alpha_m = 0.7213/(1 + 1.079/m)$ for $m \geq 128$.
Initialize m registers $M[0]$ to $M[m-1]$ to 0.

Phase 1: Aggregation.

for all $v \in S$ **do**
  $x := h(v)$
  $idx := \langle x_{31}, \ldots, x_{32-p} \rangle_2$ { First p bits of x }
  $w := \langle x_{31-p}, \ldots, x_0 \rangle_2$
  $M[idx] := \max\{M[idx], \varrho(w)\}$
end for

Phase 2: Result computation.

$$E := \alpha_m m^2 \cdot \left(\sum_{j=0}^{m-1} 2^{-M[j]} \right)^{-1}$$ { The "raw" estimate }

if $E \leq \frac{5}{2}m$ **then**
  Let V be the number of registers equal to 0.
  if $V \neq 0$ **then**
    $E^* :=$ LinearCounting(m, V)
  else
    $E^* := E$
  end if
else if $E \leq \frac{1}{30}2^{32}$ **then**
  $E^* := E$
else
  $E^* := -2^{32} \log(1 - E/2^{32})$
end if
return E^*

Summary of Programming Challenge

The programming challenge asks students to design an approximate algorithm for counting the number of distinct elements within a sliding window over a large data stream with strict memory constraints. The problem tries to frame the scenario in a realistic engineering setting where elements arrive continuously, storage is limited, and sufficiently accurate estimates may be acceptable rather than exact answers.

The assignment targets several learning objectives. First, the assignment develops an understanding of a probabilistic data structure. The solution requires a conceptual understanding of the HyperLogLog algorithm, including how it may be used to estimate cardinality with limited memory. Secondly, the assignment extends the static algorithm to a dynamic setting. While a standard HyperLogLog algorithm estimates cardinality over a static set, this programming challenge requires an adaptation of the HyperLogLog to support sliding windows. This modification requires thinking about the time dimension of data, data expiration, and aggregation. Thirdly, the assignment requires a balance of accuracy, memory usage, and performance. The assignment instructions explicitly allows approximate answers given memory constraints, and a solution requires trade-offs between estimation accuracy and efficiency. As such, a student must make design choices instead of relying on a naive, exact algorithm.

In the programming challenge, applying the basic HyperLogLog algorithm does not work for a sliding window, since the algorithm does not support deletions. As such, students must develop a workaround, such as by decomposing the window into substructures that can be combined. The model solution offers one potential workaround by dividing the sliding window into blocks of fixed sizes.

Key Ideas for Solving Programming Challenge

The key idea behind the solution is to adapt HyperLogLog, an algorithm designed for static sets, to a sliding-window stream by partitioning the stream into fixed-size blocks, each summarized by its own HyperLogLog sketch. The main challenge is handling element expiration, which may at first be difficult to see since HyperLogLog does not support deletions. In the model solution, this is addressed by discarding entire blocks as they fall outside the window and merging only the sketches that remain relevant. This approach is interesting because it requires making a decision to sacrifice exactness for efficiency like the original HyperLogLog algorithm, while remaining accessible by relying on HyperLogLog's natural mergeability rather than other advanced mathematical techniques. Sacrificing accuracy for efficiency is a difficult decision to make, and students must choose a reasonable block size that balances the two, but this trade-off is intuitive and well within the expectations of an upper-level undergraduate. The model solution runs in constant time per update on average, with query time proportional to the number of blocks times the number of registers, and uses sublinear memory relative to the stream size, making it efficient, realistic, and interesting for learning objectives.

Conclusion

HyperLogLog is an algorithm for estimating the cardinality of a multi-set, improving upon the naive approach by using a constant amount of memory and trading off exactedness. It is based on the idea that in a uniform distribution, the bits in a bitstring should be equally likely to be a 0 or 1, so by tracking the number of leading 0s in elements seen, we can make a probabilistic estimate on the number of distinct elements in the set. HyperLogLog has applications in cybersecurity, biology, and data processing. The application described in the initial paper detects the number of unique header pairs in a data stream over a network helps

detect self-propagating worms. Our programming challenge further provides a practical application of the algorithm in a situation one may see in engineering.